HW 02 — Interpolation on a Non-Uniform Grid

Problem Statement

Given a non-uniformly spaced table of $\cos(x)$ on $[0, 10]$ (101 interior-perturbed points), implement and compare three interpolation methods evaluated at 10 random test points. Repeat the comparison after thinning the table to every 2nd point (~51 points) and every 4th point (~26 points). For each method and table density, report the maximum absolute error across all test points.

The non-uniform grid is constructed by taking integer indices $0, 1, \ldots, 100$, rescaling to $[0, 10]$, and adding a random perturbation $\pm\frac{1}{3}$ of the nominal spacing to each interior point.

The Three Interpolation Methods

Method 1: Linear Interpolation — $O(h^2)$

What it is: Given the test point x_1, find the interval $[x_i, x_{i+1}]$ containing it, then linearly connect the two endpoint values:

$$y_1 = y_i + (y_{i+1} - y_i)\frac{x_1 - x_i}{x_{i+1} - x_i}$$

Implementation:

```
i = np.searchsorted(x, x1, side='right') - 1   # find left bracket index
i = int(np.clip(i, 0, n - 2))
return float(y[i] + (y[i+1] - y[i]) * (x1 - x[i]) / (x[i+1] - x[i]))
```

`np.searchsorted` performs binary search in $O(\log n)$ time to locate the bracket.

Error: The leading-order truncation error comes from the neglected curvature: $\varepsilon \approx \frac{1}{8}h^2 f''(\xi)$, where h is the local spacing. When the spacing doubles, the error grows by a factor of ≈ 4.

Method 2: Lagrange 4-Point Interpolation — $O(h^4)$

What it is: Fit a cubic polynomial through 4 nodes $x_{i-1}, x_i, x_{i+1}, x_{i+2}$ bracketing x_1 (2 nodes on each side). The interpolant is the unique degree-3 polynomial passing through all four:

$$y_1 = \sum_{k=0}^{3} y_{n_k} L_k(x_1), \qquad L_k(x_1) = \prod_{j \neq k} \frac{x_1 - x_{n_j}}{x_{n_k} - x_{n_j}}$$

L_k is the Lagrange basis polynomial: it equals 1 at x_{n_k} and 0 at all other nodes.

Implementation:

```
i = int(np.searchsorted(x, x1, side='right') - 1)
# Boundary rule: return table value near edges
if x1 <= x[1]: return float(y[1])
if x1 >= x[n-2]: return float(y[n-2])
i = int(np.clip(i, 1, n - 3))  # ensure [i-1, i, i+1, i+2] are valid
xn = x[[i-1, i, i+1, i+2]]
```

```
yn = y[[i-1, i, i+1, i+2]]

result = 0.0
for k in range(4):
    Lk = 1.0
    for j in range(4):
        if j != k:
            Lk *= (x1 - xn[j]) / (xn[k] - xn[j])
    result += yn[k] * Lk
```

Error: The truncation error is $O(h^4)$: the first neglected Taylor term involves $f^{(4)}(x)$. When spacing doubles, error grows by ≈ 16.

Boundary handling: When $x_1 \leq x[1]$ or $x_1 \geq x[n-2]$, there aren't 2 nodes on both sides, so the code falls back to the nearest table value.

Method 3: Hermite 4-Point Interpolation — $O(h^4)$

What it is: A cubic spline on the interval $[x_i, x_{i+1}]$ that matches both the function **values** and the first **derivatives** at the two endpoints. This guarantees a C^1 (continuously differentiable) interpolant — no kinks at the breakpoints.

The cubic Hermite basis (using parameter $t = (x_1 - x_i)/h$, $h = x_{i+1} - x_i$):

$$H(x_1) = (2t^3 - 3t^2 + 1)\, y_i + (t^3 - 2t^2 + t)\, h\, m_i + (-2t^3 + 3t^2)\, y_{i+1} + (t^3 - t^2)\, h\, m_{i+1}$$

where m_i and m_{i+1} are the estimated derivatives at the two endpoints.

Derivative estimation: Since the grid is non-uniform, simple central differences give only $O(h)$ accuracy. Instead, the 3-point non-uniform derivative formula (derived from the Lagrange polynomial through x_a, x_m, x_b) gives $O(h^2)$:

$$f'(x_m) = f_a \frac{x_m - x_b}{(x_a - x_m)(x_a - x_b)} + f_m \frac{2x_m - x_a - x_b}{(x_m - x_a)(x_m - x_b)} + f_b \frac{x_m - x_a}{(x_b - x_a)(x_b - x_m)}$$

For m_i, the three nodes used are x_{i-1}, x_i, x_{i+1}; for m_{i+1}, they are x_i, x_{i+1}, x_{i+2}.

Implementation:

```
def deriv3p(xa, xm, xb, fa, fm, fb):
    return (fa*(xm-xb)/((xa-xm)*(xa-xb))
          + fm*(2*xm-xa-xb)/((xm-xa)*(xm-xb))
          + fb*(xm-xa)/((xb-xa)*(xb-xm)))

mi  = deriv3p(xn[0], xn[1], xn[2], yn[0], yn[1], yn[2])
mi1 = deriv3p(xn[1], xn[2], xn[3], yn[1], yn[2], yn[3])
h = xn[2] - xn[1]
t = (x1 - xn[1]) / h
return ((2*t**3 - 3*t**2 + 1)*yn[1] + (t**3 - 2*t**2 + t)*h*mi
      + (-2*t**3 + 3*t**2)*yn[2]   + (t**3 - t**2)*h*mi1)
```

Error: Also $O(h^4)$, same order as Lagrange 4-point. In practice the error constant is different because the cubic Hermite and the Lagrange cubic use different approaches to fit the data.

Results and Analysis

Observed Error Summary

Table	Linear max err	Lagrange4 max err	Hermite4 max err
101 pts	1.6×10^{-3}	3.6×10^{-6}	1.1×10^{-5}
51 pts	6.4×10^{-3}	5.0×10^{-5}	1.1×10^{-4}
26 pts	1.8×10^{-2}	5.4×10^{-4}	7.7×10^{-4}

Convergence rates: When spacing doubles (halving the number of points from 101 to ~51): - Linear: error grows by ~$4\times$ — consistent with $O(h^2)$. - Lagrange4 and Hermite4: error grows by ~10–$14\times$ — consistent with $O(h^4)$ (the ideal factor is 16; the ratio is approximate because the grid is randomly perturbed and thinning does not exactly double every local spacing).

Why Hermite can be slightly worse than Lagrange here: Hermite requires estimating derivatives numerically (via the 3-point formula), which introduces additional approximation error. The net truncation constant can be larger than Lagrange's at the same order, especially on coarse grids where the derivative estimate is less accurate.

Log-log slope on the plot equals the order: A log-log plot of max error vs. number of table points should show slopes of -2 (linear) and -4 (both 4-point methods), confirming the theoretical orders.

Key Takeaways

- **Interpolation order = number of points used.** Linear (2 points) is $O(h^2)$; cubic methods (4 points) are $O(h^4)$. The error order is the slope on a log-log error-vs-density plot.
- **Halving the table density** multiplies the error by 2^{order}: $\times 4$ for linear, $\times 16$ for the cubic methods. This is the practical signature of the convergence order.
- **Lagrange interpolation** fits the unique polynomial through the nodes via basis polynomials L_k (each is 1 at its node, 0 at the others). Simple and accurate, but only C^0 across intervals.
- **Hermite interpolation** matches values *and* derivatives at the endpoints, producing a smooth C^1 curve. The derivatives must themselves be estimated — here via a 3-point formula.
- **Non-uniform grids need non-uniform formulas.** A naive central difference assumes equal spacing and drops to $O(h)$; the proper 3-point Lagrange-derivative formula restores $O(h^2)$.
- **Bracketing is** $O(\log n)$ via binary search (`searchsorted`); higher-order methods need special handling near the boundaries where there aren't enough neighbors.